

Pairs Trading Strategies with Cointegration

Sammaritani Daniel, 5B

June 3, 2026

Contents

1	Introduction	2
2	Part One: Data Import and Analysis	2
2.1	Importing Necessary Libraries and Chart Colors	2
2.2	Importing the Data Table	2
3	Augmented Dickey-Fuller Test	3
3.1	ADF Test Code	4
4	Quantitative Trading Strategy	4
4.1	Logarithmic Conversion	4
4.2	Cointegration and the Engle-Granger Method	4
4.3	OLS Regression and the Hedge Ratio	5
4.4	Spread Construction and Dynamic Z-Score	6
5	Sending Signals and Backtest	7
5.1	Signal Generation Algorithm	7
5.2	Backtesting the Strategy	8
5.3	Result Assignment	9
6	Results and Conclusion	11
6.1	Charts	11
6.2	Conclusion	11

1 Introduction

With this Quantitative Finance project, using a statistical arbitrage approach, we will demonstrate through a cointegration strategy, time series analysis, and time series calculation how it is possible to speculate on two correlated assets via price divergences, utilizing analysis with Python.

The strategic idea is that these two stocks, operating in the same market and being affected by the same macroeconomic factors, must move together in the long term. If for any psychological or temporary market reason the two prices diverge, the strategy involves selling the overvalued asset (which has risen too much) and buying the undervalued asset (which has lagged behind).

It is a Market Neutral strategy, meaning it is neutral relative to the market: we do not care if the stock market rises or crashes, our sole objective is to profit from the realignment of the two assets.

2 Part One: Data Import and Analysis

2.1 Importing Necessary Libraries and Chart Colors

```
1 import yfinance as yf
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import matplotlib.gridspec as gridspec
6 from statsmodels.tsa.stattools import adfuller, coint
7 from statsmodels.regression.linear_model import OLS
8 from statsmodels.tools import add_constant
9
10 BLUE = "#2563EB"
11 RED = "#DC2626"
12 GREEN = "#16A34A"
13 AMBER = "#D97706"
14 GRAY = "#687280"
15 BG = "#F9FAFB"
16 DARK = "#111827"
```

2.2 Importing the Data Table

```
1 def scarica_dati(ticker1, ticker2, start="2019-01-01",
2   ↪ end="2025-01-01"):
3   ↪ # We extract the data as strings, which will give us the
4   ↪ dataframe for our correlation
5   datiG = yf.download([ticker1, ticker2], start=start, end=end,
6   ↪ auto_adjust=True, progress=False)["Close"]
```

```

5 % Via the YF APIs we want to retrieve the price lists of the 2
   ↪ Tickers.
6 % With auto_adjust=True we have a function that adjusts the
   ↪ price excluding dividends and market splits.
7 % With progress=False we disable the loading bar.
8 datiG = datiG[[ticker1, ticker2]] # Reorder columns
9 datiG.columns = [ticker1, ticker2] # Assign names
10
11 % The ffill() function selects data with gaps (holidays,
   ↪ untraded asset) and copies the last price,
12 % while dropna() eliminates rows without data
13 datiG.ffill(inplace=True)
14 datiG.dropna(inplace=True)
15 return datiG

```

3 Augmented Dickey-Fuller Test

Before performing the ADF Test, it is necessary to discuss stationarity: a series (generally of data) is stationary when it varies according to its mean, variance, and autocovariance, whereas it is non-stationary when it has a unit root, meaning the series moves without mathematical correlation.

The test will provide a null hypothesis if the series is non-stationary, thereby proving that the stocks move by random walk and not by perfect regimes.

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \sum_{i=1}^p \delta_i \Delta y_{t-i} + \epsilon_t$$

- Δy_t is the change in the time series at time t (i.e., $y_t - y_{t-1}$).
- α is the constant, representing the starting value.
- βt is the deterministic time trend (used to understand if there is a constant tendency to rise or fall over time).
- γ is the most important parameter of the test. The test verifies if $\gamma = 0$. If $\gamma = 0$, it means the series has a unit root.
- $\sum_{i=1}^p \delta_i \Delta y_{t-i}$ is the "Augmented" part. These are the past values of the changes, inserted to clean the data from autocorrelation.
- ϵ_t is the "white noise", meaning the random error or unexpected event that cannot be predicted.

3.1 ADF Test Code

```
1 def test_adf(serie: pd.Series, nome: str) -> dict:
2     # The ADF test proves whether a series has a unit root (is
3     ↪ non-stationary)
4     risultato = adfuller(serie, autolag="AIC")
5
6     stat = risultato[0]
7     pvalue = risultato[1]
8     staz = pvalue < 0.05
9
10    if staz == True:
11        print(f"The series is stationary, with p-value ({pvalue})
12        ↪ and statistic equal to {stat}")
13        esito = "Stationary"
14    else:
15        # The more negative, the better
16        print(f"The series is non-stationary because ({pvalue}) >
17        ↪ 0.05")
18        esito = "Non-Stationary"
19
20    return {"stat": stat, "pvalue": pvalue, "esito": esito}
```

4 Quantitative Trading Strategy

4.1 Logarithmic Conversion

First, all prices are converted into logarithms to transform absolute changes into percentage changes, thereby stabilizing their variance. The logarithmic transformation of prices is defined as:

$$x_t = \ln(P_{1,t}) \quad \text{and} \quad y_t = \ln(P_{2,t})$$

where $P_{1,t}$ and $P_{2,t}$ are the historical prices of the two stocks at time t .

```
1 def test_cointegrazione(prezzi1, prezzi2, ticker1, ticker2):
2     # THEORY: We transform prices into natural LOGARITHMS.
3     # Log-prices make future returns linearly additive and stabilize
4     ↪ long-term variance.
5     log_p1 = np.log(prezzi1)
6     log_p2 = np.log(prezzi2)
7
8     test_adf(log_p1, ticker1)
9     test_adf(log_p2, ticker2)
10    # We expect False from both
```

4.2 Cointegration and the Engle-Granger Method

We must also evaluate the case of cointegration: that is, even if the 2 series are non-stationary ($\gamma = 0$), there could be a case where their linear com-

bination is stationary. This is verified via the Engle-Granger test, which confirms that this correlation is structural and not accidental.

The test verifies whether the residuals e_t of the linear combination are stationary, meaning integrated of order zero $I(0)$:

$$e_t = y_t - \alpha - \beta x_t$$

If the p-value of the test applied to e_t is lower than the significance level (typically 0.05), the null hypothesis of non-cointegration is rejected.

```

1  stat, pvalue, _ = coint(log_p1, log_p2) # We use the underscore
   ↪ because coint returns 3 values; the changes are not needed
2
3  coint_ok = pvalue < 0.05
4  if coint_ok:
5      esito = "COINTEGRATED"
6  else:
7      esito = "NON-COINTEGRATED"
8
9  print(f"Engle-Granger result for {ticker1}/{ticker2}:")
10 print(f"Statistic: {stat} | P-Value: {pvalue} -> {esito}")

```

4.3 OLS Regression and the Hedge Ratio

To estimate this non-stationarity-correlation bond, linear regression with the OLS method is used, thus isolating purely accidental relationships between the 2 stocks.

The parameters of the long-term relationship are estimated using the OLS linear regression model:

$$y_t = \alpha + \beta x_t + e_t$$

where:

- β represents the optimal Hedge Ratio, which is the hedging coefficient required to build a market-neutral portfolio;
- α indicates the constant intercept that measures the structural offset between the two log-prices;
- e_t defines the spread (or error term), whose stationarity guarantees the return to equilibrium of the system and provides the operational signals for the trading strategy.

```

1  X = add_constant(log_p2)
2  modello = OLS(log_p1, X).fit()
3
4  beta = modello.params.iloc[1]

```

```

5 alpha = modello.params.iloc[0]
6 # Estimates coefficients by minimizing the sum of squared errors
7 # Beta (Hedge Ratio): How many shares of P2 hedge 1 share of P1
8 # Alpha: The constant intercept of the regression line
9
10 print(f"OLS on Log-Prices: {alpha} | Beta: {beta} | R2 =
    ↳ {modello.rsquared}")
11 return {"p_value": pvalue, "cointegrate": coint_ok, "beta":
    ↳ beta}

```

4.4 Spread Construction and Dynamic Z-Score

Once β is extracted, the system generates the time series of the logarithmic spread, defined as the linear difference of the log-prices weighted by β :

$$\text{Spread} = \ln(\text{Price Stock 1}) - \beta \ln(\text{Price Stock 2})$$

Obviously, the spread does not remain unchanged over time; in fact, to address this, we use the `rolling(60)` function, which allows us to obtain the mean and standard deviation of the previous 60 days. In this way, we can calculate the Z-score thanks to the continuous updating of the Spread.

$$\text{Z-score} = \frac{\text{Current Spread} - \text{Rolling Mean}}{\text{Rolling Standard Deviation}}$$

The Z-Score expresses the deviation of the spread from its mean in units of standard deviations; furthermore, I eliminate the first 60 days as they lack sufficient historical data for the rolling window.

```

1 def calcola_spread_zscore(prezzi1, prezzi2, beta, finestra=60):
2     log_p1 = np.log(prezzi1) # Transform asset 1 into logarithm
3     log_p2 = np.log(prezzi2) # Transform asset 2 into logarithm
4
5     # The logarithmic spread is defined as: Spread = Log(P1) - Beta
6     ↳ * Log(P2).
7     # Since the stocks are cointegrated, this specific time series
8     ↳ will be stationary.
9     # We create a dataframe to work on.
10    df = pd.DataFrame({
11        "P1": prezzi1, "P2": prezzi2,
12        "log_P1": log_p1, "log_P2": log_p2,
13        "spread": log_p1 - beta * log_p2
14    })
15
16    # The market evolves. Mean and Standard Deviation of the spread
17    ↳ are not fixed but change slowly.
18    # We use rolling(window) functions to calculate metrics only on
19    ↳ the previous 60 days.
20    df["media_rolling"] = df["spread"].rolling(finestra).mean() %
21    ↳ Rolling mean of the spread

```

```

17 df["std_rolling"] = df["spread"].rolling(finestra).std() %
   ↪ Rolling standard deviation (spread volatility)
18
19 % The Z-Score standardizes the spread: it tells us how many
   ↪ standard deviations today's spread
20 % is away from its recent mean. Formula: Z = (Spread -
   ↪ Media_Rolling) / Std_Rolling
21 df["zscore"] = (df["spread"] - df["media_rolling"]) /
   ↪ df["std_rolling"]
22 df.dropna(inplace=True)
23 return df

```

5 Sending Signals and Backtest

5.1 Signal Generation Algorithm

The portfolio can now perform 3 functions: Buy asset 1 and short-sell asset 2, Buy asset 2 and short-sell asset 1, or not enter the market. This depends on the state of the Z-score.

- The Z-Score is below -2.0: the spread is depressed, and the algorithm buys asset 1 and short-sells asset 2.
- The Z-Score is above +2.0: the spread is excessively high, and the algorithm short-sells asset 1 and buys asset 2.
- The Z-Score varies between the parameters -2.0 and +2.0: the portfolio is empty, and there are no operations.

Operations are closed when the Z-score returns within a normality range between +0.5 and -0.5. We use `diff()` to calculate the turning points, generating buy or sell signals.

```

1 def genera_segnali(df, soglia_entrata=2.0, soglia_uscita=0.5):
2     df = df.copy()
3     df["posizione"] = 0
4     posizione_corrente = 0
5
6     for i in range(len(df)):
7         z = df["zscore"].iloc[i]
8
9         if posizione_corrente == 0: % Searching for market entries
10             if z < -soglia_entrata:
11                 posizione_corrente = 1 % LONG Entry
12             elif z > soglia_entrata:
13                 posizione_corrente = -1 % SHORT Entry
14             elif posizione_corrente == 1 and z > -soglia_uscita: % Exit
   ↪ condition from LONG position

```

```

15     posizione_corrente = 0
16     elif posizione_corrente == -1 and z < soglia_uscita: % Exit
17         ↪ condition from SHORT position
18         posizione_corrente = 0
19
20     df.iloc[i, df.columns.get_loc("posizione")] =
21         ↪ posizione_corrente
22
23     df["segnale"] = df["posizione"].diff().fillna(0)
24     return df

```

5.2 Backtesting the Strategy

We test the algorithm's strategy using data from 2019 to 2025, trading based on the previously found Z-score.

```

1 def esegui_backtest(df, ticker1, ticker2, beta):
2     df = df.copy()
3     df["r1"] = df["log_p1"].diff()
4     df["r2"] = df["log_p2"].diff()
5
6     df["r_spread"] = df["r1"] - beta * df["r2"]
7     % We take the data from the next morning because the signal is
8     ↪ generated at today's close
9     % using the shift(1) function
10    df["r_strat"] = df["posizione"].shift(1) * df["r_spread"]
11    df["r_strat"].fillna(0, inplace=True) % Remove the first value
12    ↪ (NaN)
13    df["r1"] = df["r1"].fillna(0) % Remove NaNs from the series as
14    ↪ well
15
16    % We use the cumulative sum of Log-returns and then apply the
17    ↪ exponential
18    % to transform data into multipliers
19    df["Cum_strat"] = np.exp(df["r_strat"].cumsum())
20    df["buy-and-hold_strat"] = np.exp(df["r1"].cumsum())
21
22    ngiorni = len(df)
23    strat_return = df["Cum_strat"].iloc[-1] - 1 % Total pairs return
24    bh_return = df["buy-and-hold_strat"].iloc[-1] - 1 % Classic
25    ↪ total return
26
27    % Now we extract the daily mean and annualized volatility of the
28    ↪ strategy to find the Sharpe ratio
29    mu_anno = df["r_strat"].mean() * 252 % (trading days)
30    vol_annua = df["r_strat"].std() * np.sqrt(252)
31
32    if vol_annua > 0:
33        sharpe = mu_anno / vol_annua

```



```

28     else:
29         sharpe = 0
30
31     % We find the maximum drawdown of the portfolio
32     cum = df["Cum_strat"]
33     rolling_max = cum.cummax()
34     drawdown = (cum - rolling_max) / rolling_max
35     max_dd = drawdown.min()
36
37     n_trade = (df["segnale"].abs() > 0).sum()
38
39     print(f"Quantity of trades: {n_trade}")
40     print(f"Pairs trading return: {strat_return}")
41     print(f"Buy & Hold return: {bh_return}")
42
43     % Save metrics
44     df.attrs["metriche"] = {
45         "total return": strat_return,
46         "buy-and-hold": bh_return,
47         "sharpe": sharpe,
48         "max-drawdown": max_dd,
49         "n-trade": n_trade
50     }
51     return df

```

5.3 Result Assignment

At this point, we assign values to our variables so that we can make our strategy profitable. We choose 2 listed companies from the same highly correlatable sector, such as fossil fuels; for example, CVX (Chevron) and XOM (Exxon Mobil).

```

1 def main():
2     TICKER1 = "CVX"
3     TICKER2 = "XOM"
4
5     dati = scarica_dati(TICKER1, TICKER2, "2019-01-01",
6         ↪ "2024-01-01")
7
8     % PHASE 2: Execution of statistical analysis and cointegration
9     ↪ tests
10    risultato_stat = test_cointegrazione(dati[TICKER1],
11    ↪ dati[TICKER2], TICKER1, TICKER2)
12    beta = risultato_stat["beta"] % Saves the optimal coefficient
13    ↪ extracted from the OLS regression
14
15    % PHASE 3: Construction of the historical spread and
16    ↪ normalization into rolling Z-Score

```

```

12     df = calcola_spread_zscore(dati[TICKER1], dati[TICKER2], beta,
    ↪     finestra=60)
13
14     % PHASE 4: Empirical verification. If previous calculations are
    ↪     correct, the spread MUST be stationary.
15     print("\nADF Test on the logarithmic (stationary) SPREAD:")
16     test_adf(df["spread"], "spread")
17
18     % PHASE 5: Chronological generation of operational signals based
    ↪     on strategy rules
19     df = genera_segnali(df, soglia_entrata=2.0, soglia_uscita=0.5)
20
21     % PHASE 6: Historical portfolio simulation (Backtest) and Sharpe
    ↪     Ratio / Drawdown calculation
22     df = esegui_backtest(df, TICKER1, TICKER2, beta)
23
24     % PHASE 7: Generation of the final graphic dashboard
25     plot_risultati(df, TICKER1, TICKER2, df.attrs["metriche"])
26
27     if __name__ == "__main__":
28         main()

```

6 Results and Conclusion

6.1 Charts

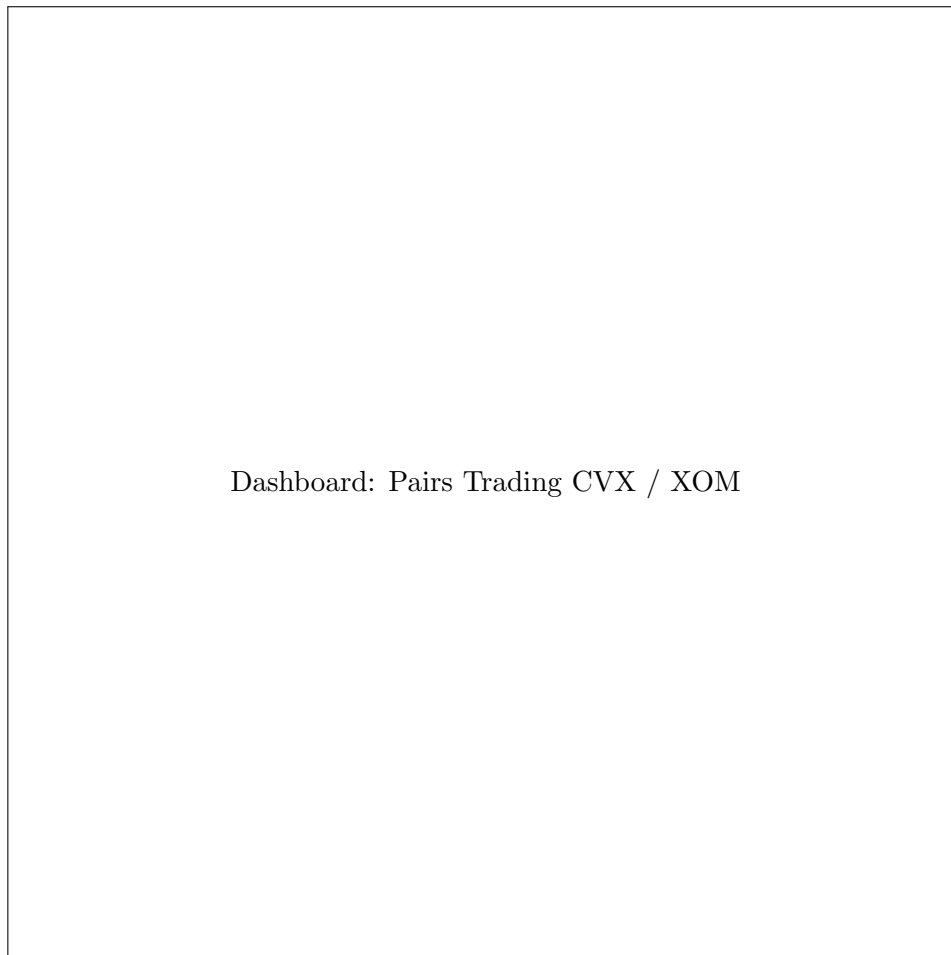


Figure 1: Strategy Results (Normalized Prices, Logarithmic Spread, Z-Score, and Cumulative Returns with metrics: Sharpe: 0.64, Max DD: -22.2%, Return: +56.9%)

6.2 Conclusion

From this strategy backtesting, we can observe how our strategy achieved a better performance compared to the Buy and Hold of CVX, with a spread of approximately 7%, without taking into account the impact of transaction fees on the strategy.

From charts 1 and 2, it is possible to see the correlation of the assets, which tend to meet at non-stationary points relative to one another. In

Chart 3, we can observe the Z-score and the movements of the strategy based on the value of the previous variable.

Meanwhile, in chart 4, we can see the stability of the strategy, which was highly profitable during the COVID period (we managed to profit from the oil crisis); however, from 2022 onward, the strategy lost efficacy, flattening out almost entirely to match the values of the buy & hold approach.